
Particle Filter for Single Object Tracking

Sidisha Shyam Barik
University of Massachusetts Amherst
Amherst, Massachusetts
sbarik@umass.edu

Abstract

1 This report details the implementation of particle filter based tracking for a single
2 vehicle (car, bus, or truck) within an indoor environment. Vehicle localization is
3 achieved using measurements from four cameras. The core of the work involves
4 the implementation of a Bootstrap Particle Filter with Constant Velocity (CV)
5 and Constant Turn Rate and Velocity (CTRV) motion models. Enhancements
6 such as measurement gating, a birth-and-death model for track management, a
7 Differentiable Particle Filter for parameter learning, and an Auxiliary Particle Filter
8 to improve proposal distribution are explored and discussed. PFs with constraints
9 are also explored. The project evaluates the performance of these methods and
10 discusses their respective advantages and challenges in the context of the given
11 tracking problem.

12 1 Introduction

13 This project focuses on the problem of tracking a single vehicle (a car, bus, or truck) operating
14 within a confined indoor room. The environment is monitored by four cameras (nodes), which
15 provide measurements regarding the vehicle's position. Each camera is associated with a model
16 that yields an estimated mean (μ) and standard deviation (σ) of the vehicle's expected location at
17 each discrete timestep. Up to four measurements can be available at any given time. The primary
18 objective of this work is to develop a robust system for generating the vehicle's trajectory using
19 these multi camera measurements. Particle Filters (PFs) were chosen as the foundational algorithm
20 for this task. Unlike Kalman Filters and its variants (Extended Kalman Filter, Unscented Kalman
21 Filter), PFs do not rely on the assumption that the underlying state or noise distributions are Gaussian.
22 This characteristic allows PFs to effectively model and emulate distributions that may be highly
23 non-linear, non-Gaussian, or even multi-modal, which can be advantageous in complex tracking
24 scenarios. This report details the implementation of a Bootstrap Particle Filter, explores different
25 motion models, and investigates several enhancements including a birth-and-death model for track
26 management, differentiable particle filters for end-to-end learning, and auxiliary particle filters for
27 improved sampling efficiency. PFs with constraints is also explored.

28 2 Literature Review

29 The following works were reviewed to understand the problem being explored.
30 Sequential Monte Carlo techniques for tracking applications are detailed by Samarjit Das in this work
31 (1). This work provides an in-depth understanding of how Particle Filters (PFs) estimate the posterior
32 probability distribution of a state by utilizing Monte Carlo methods, which is central to the tracking
33 problem addressed in this project.
34 The work by Karkus et al., Particle Filter Networks with Application to Visual Localization(2),
35 provides insights on how to make Particle Filters end-to-end differentiable. This is directly used

36 while trying to make PFs differentiable with the aim of learning the filter parameters through
37 backpropagation.

38 The paper Tracking Multiple Objects with Particle Filtering by Hue et al. (3) was reviewed to
39 understand techniques for multi-target tracking, particularly the use of Gibbs Sampling for jointly
40 estimating the joint state, data associations, and mixture probabilities. Similarly, the work MCMC-
41 Based Particle Filtering for Tracking a Variable Number of Interacting Targets (4), introduced some
42 interesting ideas for PFs dealing with interacting targets by incorporating a Markov Random Field
43 motion model. This concept of modeling interactions was considered during the exploration of
44 constraining particle filter predictions within map boundaries.

45 The book Probabilistic Machine Learning (5) provided an in-depth understanding of PFs along with
46 the mathematical foundations required to understand the above works. The book Bayesian Filtering
47 and Smoothing (6) also offered a deep dive into Particle Filters and their variations, including the
48 Auxiliary Particle Filter (APF) which was implemented and investigated in this project.

49 The specific problem setting and architecture for this project can be found in Heteroskedastic
50 Geospatial Tracking with Distributed Camera Networks (7). This paper address the challenge of
51 geospatial object tracking using data from a distributed camera network by providing an end-to-end
52 architecture for vehicle tracking using a set of heteroskedastic geospatial detection (HGD) models
53 and a geospatial tracker based on a multi-observation Kalman filter. The current work builds directly
54 upon this foundation by replacing the Kalman Filter with various Particle Filter implementations and
55 enhancements.

56 3 Dataset

57 The tracking environment consists of a room equipped with four cameras (nodes). These cameras
58 provide observations of the vehicle. At each timestep, each camera can provide a measurement
59 $z_i = (\mu_i, \sigma_i)$ as mentioned above in the paper (7), representing the estimated mean and standard
60 deviation of the vehicle's observed position (typically x, y coordinates). Thus, at any timestep t, we
61 can receive a set of up to m measurements $z_{k,1}, \dots, z_{k,m}$. The detections are extracted at a resolution
62 corresponding to a time interval = 1/15 seconds. This δ_t is critical for the motion models.

63 4 Particle Filters

64 Particle Filtering algorithm is used for approximating the posterior distribution of a state in a
65 dynamical system, given a sequence of observations. Its strength lies in its ability to represent
66 arbitrary probability distributions, making them suitable for non-linear and non-Gaussian systems.

67 The fundamental particle filter implemented in this project is the Bootstrap Particle Filter. In this
68 filter, the importance distribution (proposal distribution) is chosen to be the state transition model
69 (dynamics model), $p(x_k | x_{k-1})$. The general algorithm involves initialization, prediction, update, and
70 resampling steps. A key aspect of the implementation is its handling of missing measurements if no
71 measurements are received at a given timestep, only the prediction step is performed, propagating
72 particles based on the dynamics model.

73 The algorithm implemented for Bootstrap Particle Filer 1 is as mentioned in Algorithm 11.9 in (6)

74 4.1 Motion Models

- 75 • Constant Velocity Model - The Constant Velocity model assumes that the vehicle moves
76 with a constant velocity between timesteps, subject to some process noise. The state vector
77 here is defined as $[x_1, x_2, v_{x_1}, v_{x_2}]$ where (x_1, x_2) represent the 2D position position and
78 (v_{x_1}, v_{x_2}) represents the 2D velocities.
- 79 • Constant Turn Rate Model - The Constant Turn Rate Model assumes that the state is
80 described not only by its coordinates and linear velocity but also orientation and angular
81 velocity. Since the model allows for changes in orientation, it is expected to better capture
82 the curving trajectories compared to the CV model. The state vector here is defined as
83 $[x_1, x_2, \psi, s, \omega]$ where (x_1, x_2) is the 2D position, s is the speed, ψ is the heading angle
84 and ω is the the yaw rate.

Algorithm 1 Bootstrap Particle Filter

Given a sample set from the previous time step $\{x_{k-1}^{(i)} : i = 1, \dots, N\}$:

Predict:

For each particle $i = 1, \dots, N$:

Draw a new state $x_k^{(i)}$ from the dynamic model (proposal distribution):

$$x_k^{(i)} \sim p(x_k | x_{k-1}^{(i)}).$$

Update:

For each particle $i = 1, \dots, N$:

Calculate the unnormalized importance weight:

$$w_k^{*(i)} = p(y_k | x_k^{(i)}).$$

Normalize the weights to sum to unity:

$$w_k^{(i)} = \frac{w_k^{*(i)}}{\sum_{j=1}^N w_k^{*(j)}}.$$

Resample:

If the effective number of particles:

$$n_{eff} = \frac{1}{\sum_{i=1}^N w_k^{(i)2}}$$

is too low, perform resampling based on the weights $\{w_k^{(i)}\}_{i=1}^N$ to obtain a new set of N particles $\{x_k^{(i)'}\}_{i=1}^N$.

85 4.2 Implementation

86 4.2.1 Initialization

87 The filter is initialized with $N=1000$ particles. These particles are typically sampled around the
88 first available measurement. An additive Gaussian noise, scaled by a parameter, is applied to the
89 initial particle states to create a spread. The weights w_k^i for each particle i are initialized uniformly:
90 $w_0^i = 1/N$

91 4.2.2 Predict

92 In the predict step, each particle x_{k-1}^i is propagated forward in time according to the chosen dynamics
93 model (CV or CTRV) to obtain a predicted state : $x_k^{(i)} \sim p(x_k | x_{k-1}^{(i)})$

94 This involves applying the deterministic part of the motion model and then adding process noise. The
95 process noise covariance is computed based on the dynamics model and δ_t . For N particles, we sample
96 N noise vectors. In this implementation, `torch.distributions.MultivariateNormal.sample` is used
97 for drawing samples from a multivariate normal distribution with the mean as the deterministically
98 propagated particle state and the computed process noise covariance.

99 Here, it is also ensured that the angles are wrapped between $-\pi$ and π using `atan2`.

100 4.2.3 Update

101 Following the prediction step, the particle weights are updated based on the new measurements.
102 For the Bootstrap Particle Filter, the weight update for particle i is proportional to the likelihood
103 of the measurement given the particle's state: $w_k^{*(i)} = p(y_k | x_k^{(i)})$. The likelihood is computed
104 by assuming it to be Gaussian with a measurement noise covariance R_k provided by the camera
105 model, centered around the expected measurement derived from the particle state $x_k^{(i)}$ and computed
106 for each measurement z_k . To ensure numerical stability, weight computations were performed in
107 the log domain, employing the log-sum-exp trick for normalization. Initially, a sequential update
108 approach was implemented, where each of the M measurements at a timestep leads to a separate
109 update iteration. Subsequently, a batch update method was also implemented. In batch update, the
110 weight incorporates the likelihood from all available measurements and the weights are updated by

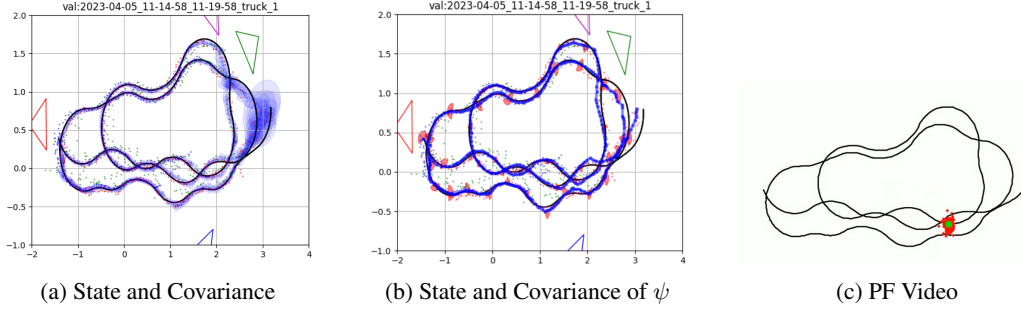


Figure 1: Visualization for Particle Filter

combining all measurement likelihoods. In log space $\log(w_k^i) = \sum_{m=1}^M \log(p(y_k^m | x_k^{(i)}))$. This batch approach is found to be computationally faster.

4.2.4 Resampling

Particle degeneracy, where a few particles acquire most of the weight, is a common issue. To mitigate this, resampling is performed when the Effective Sample Size (ESS) drops below a threshold, typically $\frac{N}{2}$. ESS is calculated as: $n_{eff} = \frac{1}{\sum_{i=1}^N w_k^{(i)2}}$. If $n_{eff} < \frac{N}{2}$, particles are resampled with replacement from the current set of particles, where each particle is drawn with probability equal to its weight. This is achieved by sampling from a multinomial distribution. After resampling, all particle weights are reset to $\frac{1}{N}$.

Following the completion of the predict, update, and (if performed) resampling steps for a given timestep, the final state estimate of the vehicle is extracted. This is achieved by computing the weighted mean of the particle states. The uncertainty associated with this estimate is obtained by calculating the weighted covariance of the relevant particle state components. For the CTRV model, a weighted circular mean and covariance is taken.

4.3 Modifications/Enhancements

4.3.1 Gating

To improve robustness against outliers, a gating mechanism was implemented. Before the update step, measurements are validated against the predicted distribution of the particles. A gate is defined around the weighted mean and weighted covariance of the particles after the predict step. An ellipsoidal gate corresponding to a probability mass of 99% likelihood under a Gaussian assumption for the predicted measurement is used. Only measurements falling within this gate are considered for the weight update step.

4.3.2 Visualizations for Diagnostics and Assessment

Several visualization techniques were employed to assess filter performance and aid in debugging:

- **State and Covariance Visualization:** The estimated state (x_1, x_2) and its corresponding covariance ellipse are plotted at each timestep as the track progresses. This provides an intuitive understanding of the filter's certainty.
- **Psi (Orientation) Visualization:** For the CTRV model, in addition to the (x_1, x_2) track and covariances, the orientation psi is visualized. This is represented as a wedge or oriented ellipse, where the center indicates the mean angle and the angular width represents its uncertainty (derived from the weighted circular variance of ψ).
- **Particle Filter Video:** A video was generated to show the behavior and distribution of the entire particle cloud over time as the track progresses. This was particularly helpful for debugging particle spread, degeneracy, and resampling effects.

An example of the above visualizations is shown in Figure 1.

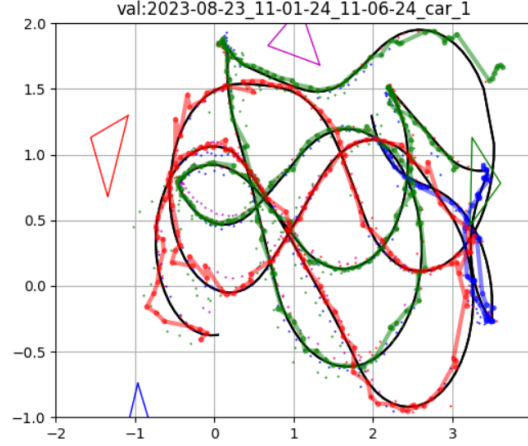


Figure 2: Birth and Death Model Performance for a car. It can be seen that when the detections are not present, the track score is reduced and a new track is initialized once detections are observed again.

5 Birth and Death Model

In scenarios with sparse detections or extended periods of time where measurements are absent, the predicted track can significantly deviate from the ground truth, depending on the motion model used. The below figure 2 shows the performance of the model on a car trajectory. To manage tracks under such conditions, an object birth and death model was implemented. The implemented algorithm is as follows:

- **Track Initiation:** A new track is initiated when new measurements are received that cannot be associated with existing active tracks.
- **Track Maintenance and Scoring:** A score is maintained for each track. This score is incremented when associated measurements are received consecutively.
- **Track Termination :** The track score is decremented if no measurements are associated with the track for a certain number of timesteps. If the score drops below a predefined threshold, the track is terminated. A new track is initiated if relevant measurements appear later.

Since, there are hard thresholds on when to initiate and drop a track, this approach is inherently a non-differentiable operation making it difficult to train through. Optimizing such a system would require careful consideration of various factors in a suitable loss function, including how long a particular track is being tracked, the number of accurately predicted tracks, the accuracy of state estimates within maintained tracks, and penalties for false tracks or missed tracks.

6 Differentiable PF

Following the implementation of the standard PF, an end-to-end Differentiable Particle Filter was developed. This enables the learning of internal filter parameters, such as the process noise standard deviations (std_acc for CV, or std_accl and std_accy for CTRV) through backpropagation. It also facilitates the integration of the PF with the camera model generating μ and σ for end-to-end training of the entire pipeline. To achieve differentiability, two key modifications(2) were made to the standard PF algorithm:

6.1 Sampling

Sampling inherently is not a differentiable function. In order to be able to differentiate through sampling, we perform the re-parameterization trick.

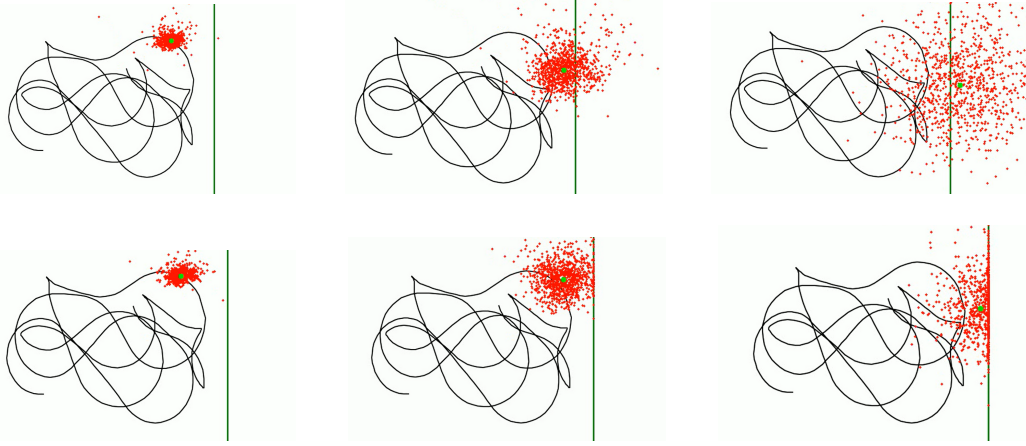


Figure 3: A demonstration of PF working under constraints. The top 3 images denote how the particles move when they are unbounded. It can be seen that though there is a wall (green wall) on the right, the particles spread over the wall. The bottom 3 images denote how the particles move when they are bounded. It can be seen that when bounded, the particles are restricted to only be within the defined boundaries.

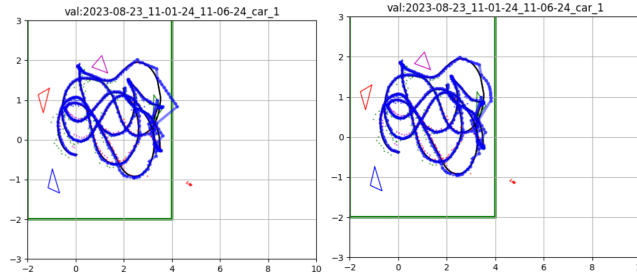


Figure 4: A demonstration of PF working under constraints. The left plot denotes the performance of the Particle Filter without any constraints. The right plot denotes the performance of the Particle Filter with constraints. It can be seen that since the particles are restricted, the predicted track is closer to the ground truth.

174 6.2 Soft Resampling

175 In the update step, there is a resampling step. Post the resampling step, the weights of the particles
 176 are set as constant $\frac{1}{N}$. Due to this, the gradients produced are 0 and it is difficult to back propagate
 177 through. In order to overcome this, Soft resampling technique is used that does not sample directly
 178 from a Multinomial distribution of weights but adds a uniform distribution to it as well. Thus, the
 179 weights post resampling are not constant but rather dependent on the initial weights and the alpha
 180 parameter that controls how much of the original multinomial distribution we want to consider. This
 181 allows the gradients to flow through the weights.

182 6.3 Differentiable PF Training

183 Using the differentiability of the implemented DPF, experiments were conducted to learn a few
 184 parameters :

- 185 • The standard deviations of acceleration components in the motion models (*std_acc* in the
 186 CV model, or *std_accl* and *std_accy* in the CTRV model) were observed to have an impact
 187 on the alignment of the predicted track with the ground truth.

- The measurement uncertainty provided directly by the camera model often appeared to be overly confident. Introducing a learnable scaling factor for this synthetic measurement uncertainty was hypothesized to improve the filter performance.

6.3.1 Training Methodology:

The Differentiable PF was trained to learn the motion model noise parameters and the measurement uncertainty scaling factor.

- **Loss Function:** The Negative Log Likelihood (NLL) loss was employed. This loss evaluates how well the ground truth state $x_{gt,k}$ at timestep k is explained by the Gaussian distribution defined by the DPF’s estimated state (weighted mean $\hat{x}_k$) and estimated covariance Σ_k . The NLL loss for a single timestep k is given by:

$$\mathcal{L}_k = -\left(\frac{1}{2}(x_{gt,k} - \hat{x}_k)^T \Sigma_k^{-1} (x_{gt,k} - \hat{x}_k) + \frac{1}{2} \log |\det(\Sigma_k)| + \log(2\pi)\right) \quad (1)$$

The total loss for a track was the sum of NLL losses over all timesteps: $\mathcal{L}_{track} = \sum_{k=1}^T \mathcal{L}_k$.

- **Optimization:** Backpropagation was performed once at the end of each complete track to compute gradients of $\mathcal{L}_{track}$ with respect to the learnable parameters. The Stochastic Gradient Descent (SGD) optimizer was used for parameter updates.

Few challenges were encountered during the training process:

- **Gradient Explosion:** Initial training attempts sometimes suffered from exploding gradients, leading to unstable updates. Gradient clipping was explored as an initial mitigation strategy.
- **Numerical Stability:** A more effective solution for stabilizing the backpropagation process, particularly issues arising from matrix inversion (Σ_k^{-1}) of the covariance matrix, was the addition of a small diagonal jitter. A small positive value ϵ was added to the diagonal elements of Σ_k . This improved the conditioning of the matrix and made the training more numerically stable.

6.3.2 Observations:

Despite these efforts, the training procedure did not consistently lead to convergence of the learnable parameters. This could be due to:

- **Rugged Loss Landscape:** The loss surface for PF parameters is likely rugged, making it challenging for SGD to find a good minimum.
- **Monte Carlo Noise:** Particle Filters are inherently stochastic due to the Monte Carlo sampling involved in prediction and resampling. This randomness introduces noise into the loss computation and gradient estimates, which can hinder the convergence of gradient-based optimizers. Overcoming this Monte Carlo variation would need a very large number of particles or more advanced gradient estimation techniques.

Further investigation into the above is required for more effective training.

7 Auxiliary PF

During experiments with both CV and CTRV models using $N=1000$ initial particles, it was observed that the Effective Sample Size (ESS) often dropped to approximately 50% of the total particle count ($N/2$) after the update step. This indicates that only about half of the particles effectively contribute to the posterior estimate at each update, suggesting potential sample impoverishment or a suboptimal proposal mechanism.

This significant drop in ESS can often be attributed to the proposal distribution used in the Bootstrap Particle Filter. The Bootstrap filter uses the dynamics model $p(x_k|x_{k-1})$ as its proposal distribution. This means particles are proposed (predicted) without considering the most recent measurement z_k . If the dynamics model poorly predicts the region where the new measurement lies, many propagated

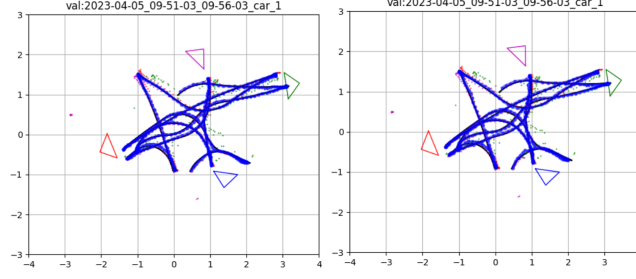


Figure 5: PF vs APF 1000 particles. The performance of both PF(left) and APF(right) is found to be similar with 1000 particles.

particles may end up in regions of low likelihood $p(z_k|x_k^i)$, leading to low weights and a drop in ESS after the update.

The Auxiliary Particle Filter aims to address this by incorporating the current measurement z_k into the proposal mechanism, effectively sampling from a distribution that is closer to the optimal importance distribution $p(x_k|x_{k-1}, z_k)$. The idea is to predict where the particles might move, weight the particles based on their measurement likelihood and then propagate those particles that are more likely to be around the measurements obtained. The algorithm implemented 2 is from Algorithm 11.12 in (6). By first selecting promising parent particles based on the current measurement, the APF aims to ensure that the propagated particles x_t^i are more likely to be in regions of high measurement likelihood, thus improving ESS and overall filter performance.

Algorithm 2 Auxiliary Particle Filter

Draw samples $x_0^{(i)}$ from the prior and set $w_0^{(i)} = \frac{1}{N}$ for $i = 1 \dots N$.

For each $k = 1 \dots T$ do the following:

 Generate μ_k^i from $p(x_k|x_{k-1}^i)$

 Compute the first-stage weights

$$\lambda^i = w_{k-1}^i p(y_k|\mu_k^i)$$

 and normalize them to sum to unity

 Resample $\{., \lambda^{(i)}\}$ to get a new set of indices $\kappa^{(i)}$

 Draw new samples from the dynamic model

$$x_k^i \sim p(x_k|x_{k-1}^{\kappa^i})$$

 Compute new weights

$$w_k^i = \frac{p(y_k|x_k^i)}{p(y_k|\mu_k^{\kappa^i})}$$

 and normalize them to sum to unity.

8 PF with constraints

Another advantage of Particle Filters is their flexibility in incorporating environmental information, such as known boundaries or maps, directly into the filtering process. This can prevent particles from propagating into physically impossible regions, thereby improving tracking accuracy and robustness, especially when measurement data is sparse or noisy. To explore this capability, experiments were conducted to constrain particles within the known boundaries of the room. Two approaches were explored:

8.1 Particle Constraint during Prediction:

This method enforces boundary constraints directly after the prediction step. Once particles are propagated according to the motion model, their new states are checked against the predefined boundary conditions (like the walls of the room).

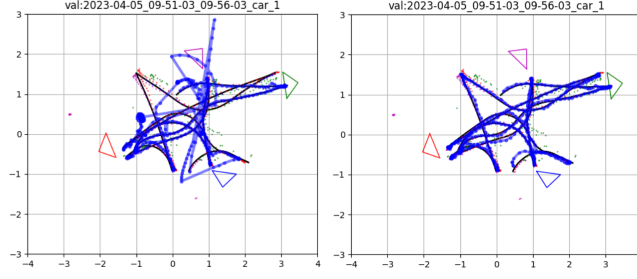


Figure 6: PF vs APF 150 particles. It can be seen that the performance of PF with 150 particles has significantly degraded (left). On the other hand, with the same number of particles, APF is able to predict a more accurate track(right).

- If a predicted particle $x_k^{(i)}$ is found to be outside the valid area, it is rejected.
- An attempt is then made to resample a new state for this particle, ensuring the new sample lies within the boundary. This resampling process is typically repeated up to a predefined maximum number of attempts to avoid potential infinite loops (e.g., if the motion model consistently pushes particles out of bounds).
- If, after the maximum attempts, a valid in-bounds particle cannot be generated through resampling, the particle's state is clamped to the point on the boundary.
- If a particle's coordinate (e.g., x) is clamped to a boundary, its corresponding velocity component ($\dot{x}$) is set to zero. This prevents the particle from maintaining a velocity component that would immediately drive it out of bounds again, effectively making it slide along the boundary if its motion in the other dimension is still valid.

This approach is particularly helpful in scenarios where measurements are absent for extended periods. Without constraints, particles are propagated based on the predict step and this might drift significantly and eventually move beyond the boundaries. The above approach ensures particles remain within plausible regions. A demonstration of this approach can be seen in Figures 3 and 4.

8.2 Weight Modification during Update:

Another approach involves penalizing particles that fall outside the permitted boundaries by adjusting their weights during the update step.

- After the prediction step, but before the final weight normalization, particles are checked against the boundary conditions.
- If a particle $x_k^{(i)}$ is determined to be outside the valid area, its weight $w_k^{(i)}$ is significantly reduced.
- The weights are then normalized as usual.

By drastically lowering their weights, out-of-bounds particles have a very low probability of being selected during the resampling phase, effectively causing them to die out over time. This method can be particularly useful for handling situations where noisy measurements might suggest a vehicle location that is slightly outside the known map boundaries, allowing the filter to discard such particles.

Table 1: PF - NLL, RMSE by Vehicle Type

	Bus		Truck		Car	
	NLL	RMSE	NLL	RMSE	NLL	RMSE
CV	-4.3875	0.0288	-3.2684	0.0491	-1.5648	0.0997
CTRV	-4.3355	0.0305	-3.4379	0.0451	-1.7574	0.1001

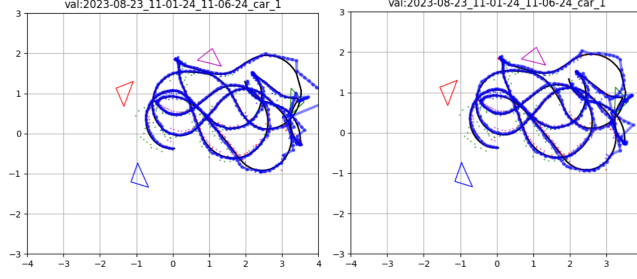


Figure 7: A demonstration of PF with CV(left) and CTRV(right) motion model. In this scenario, since there are many missed detections towards the turn, the CV model continues predicting linearly. However, the CTRV model is able to turn back onto track by assuming a constant turn rate when it loses detections.

Table 2: APF - NLL, RMSE by Vehicle Type

	Bus		Truck		Car	
	NLL	RMSE	NLL	RMSE	NLL	RMSE
CV	-3.7657	0.0286	-3.4883	0.0440	5.6464	0.1287
CTRV	-3.3033	0.0421	-3.5330	0.0442	-0.1245	0.1387

9 Experiments

The Particle Filter as well as Auxiliary Particle Filters were implemented and their performances can be seen in Tables 1 and 2. The birth and death model was also implemented as mentioned in Section 5. PF with constraints was also explored as mentioned in Section 8.

It can be seen that both CTRV and CV with PF lead to similar results 1. However, we can visually see in Figure 7 that in some cases like for a particular car scenario, when the detections are not observed, the CTRV model is able to follow the track more accurately when compared to the CV model due to its ability to account for a constant turn rate.

The results for APF are similar to the ones observed for PF 2. A high NLL is observed for the car due to a particular scenario with a lot of missed detections resulting in the CV motion model not being able to accurately predict and leading the prediction to go off-track. The performances overall seem to be as was observed with the Particle Filter.

A comparison of Particle Filter with Auxiliary Particle Filter is shown in Table 3. It is observed that the APF is able to maintain its performance even with lower number of particles. Whereas, the Particle Filter's performance degrades when we use lower number of particles. This shows that the APF in general can perform better than PF while running with lower number of particles thus reducing the computation cost. The same can be viewed in Figures 5 and 6

Table 3: APF vs PF for different number of particles (N)

	1000 Particles			500 Particles			250 Particles			150 Particles		
	NLL	RMSE	ESS	NLL	RMSE	ESS	NLL	RMSE	ESS	NLL	RMSE	ESS
PF	-2.3629	0.0651	403	-2.3810	0.0661	184	-1.9601	0.0719	89	5.6760e+34	0.8111	53
APF	-2.4933	0.0801	943	-2.4599	0.0794	471	-2.3547	0.0854	235	-2.1641	0.0831	141

References

- [1] S. Das, “Sequential monte carlo techniques and bayesian filtering: Applications in tracking,” Department of Computer Science, Iowa State University, Technical Report, dec 2008, submitted to the Department of Computer Science, Iowa State University.
- [2] P. Karkus, D. Hsu, and W. S. Lee, “Particle filter networks with application to visual localization,” 2018. [Online]. Available: <https://arxiv.org/abs/1805.08975>
- [3] C. Hue, J.-P. Le Cadre, and P. Perez, “Tracking multiple objects with particle filtering,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 38, no. 3, pp. 791–812, 2002.
- [4] Z. Khan, T. Balch, and F. Dellaert, “Mcmc-based particle filtering for tracking a variable number of interacting targets,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 11, pp. 1805–1819, 2005.
- [5] K. P. Murphy, *Probabilistic Machine Learning: Advanced Topics*. Cambridge, MA: The MIT Press, 2023.
- [6] S. Särkkä and L. Svensson, *Bayesian Filtering and Smoothing*, 2nd ed. Cambridge: Cambridge University Press, 2023.
- [7] C. Samplawski, S. Fang, Z. Wang, D. Ganesan, M. Srivastava, and B. M. Marlin, “Heteroskedastic geospatial tracking with distributed camera networks,” in *Proceedings of the Thirty-Ninth Conference on Uncertainty in Artificial Intelligence*, ser. Proceedings of Machine Learning Research, R. J. Evans and I. Shpitser, Eds., vol. 216. PMLR, 31 Jul–04 Aug 2023, pp. 1805–1814. [Online]. Available: <https://proceedings.mlr.press/v216/samplawski23a.html>